

MATH 566: Lecture 13 (10/06/2026)

- Today:
- * label correcting algo
 - poly time implementation
 - * practice midterm

Complexity of the modified label correcting algorithm

- * $d(j)$ is updated at most $2nC$ times. → $-nC \leq d(j) \leq nC$
- * After update of $d(j)$, j is added to LIST, and arcs in $A(j)$ are scanned afterward.
- * Total # arc scans $\leq \sum_{j \in N} 2nC |A(j)|$.
outarcs of j

⇒ Modified label correcting algorithm runs in $O(mnC)$ time.

→ pseudopolynomial time algorithm

We have come from an exponential time algo ($O(2^n)$) to a pseudopolynomial time algo (pseudopolynomial due to the dependence on C , rather than $\log C$). But we now present an implementation that runs in polynomial time — $O(mn)$!

FIFO Implementation of the label correcting algorithm

- * "Pass": Scan all arcs in A , update $d(j)$ if $d(j) > d(i) + c_{ij}$.
- * Do n passes, or stop if no $d(j)$'s change in a pass (whichever comes first).
- * Maintain LIST as a FIFO queue.

In fact, we don't have to examine all arcs in A in each pass! We can take a node i from top/front of the LIST, and examine only its outarcs, i.e., the arcs in $A[i]$. If we update $d(j)$ for any arc $(i,j) \in A[i]$ and $j \notin \text{LIST}$, then we add j to the back of the LIST. See AMO Fig 5.5 (in pg 141) for pseudocode.

AMO Theorem 5.3 The FIFO label correcting algorithm solves SP in $O(mn)$ time, or else shows that there is a negative cycle.

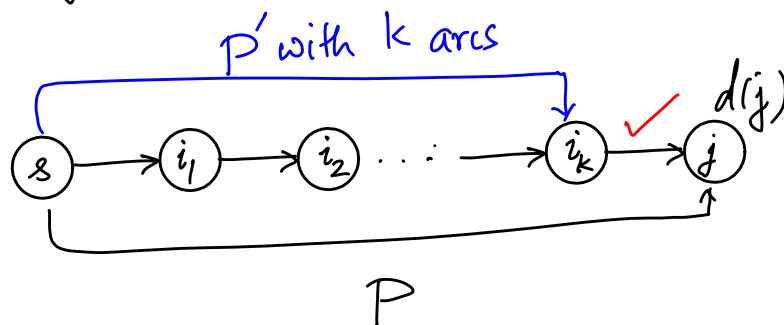
Proof Each pass : $O(m)$ time (at most m updates).

Need to show the algo will run correctly in n passes.

Claim At the end of the k^{th} pass, the algorithm computes SP distances for all nodes whose SP from s has k or fewer arcs.

We use induction to prove this claim. Assume it holds for k passes.

Consider node j .



P' is an SP from s to i_k with k arcs. By induction assumption, after k passes, we will have

$$d(i_k) = \sum_{(i,j) \in P'} c_{ij}$$

In the $(k+1)$ -st pass, we update $d(j)$ to $d(i_k) + c_{i_k,j}$. Hence $d(j)$ will be the SP distance from s to j with $k+1$ arcs.
 $\Rightarrow$ The claim holds.

Note that if $d(j)$ is not updated in the $(k+1)$ -st pass, node j will (trivially) be not a candidate to consider here.

Since any SP from s to j has at most $(n-1)$ arcs, the $d(j)$'s are optimal after $(n-1)$ passes. But if there is an update in the n th pass, there exists a negative cycle. Else, the SP problem is solved.

Maintaining LIST as FIFO queue and examining $A(i)$ of only nodes i that see changes in $d(i)$ makes the run more efficient than the $O(mn)$ worst-case running time. $\square$

If we maintain LIST as a LIFO queue, it might work better in practice, especially on large instances. But we need the FIFO handling to prove the $O(mn)$ running time.

Detecting Negative Cycles

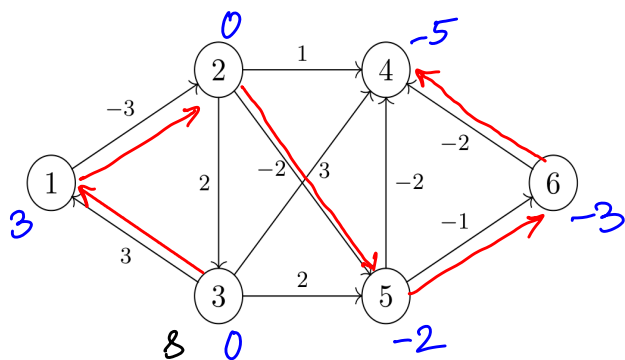
1. Stop if $d(j) \leq -nC$.
2. Run the FIFO label correcting algorithm, and stop if a node is scanned at least n times.
3. Keep track of the # arcs in an SP from s to $j, j \in N$, and stop if any SP has more than $(n-1)$ arcs.

Review for Midterm

134

For Problems 1-3, just eyeball solutions! No proof of correctness is required, nor are any evidence of use of any algorithms.

3.



$d(i)$

i

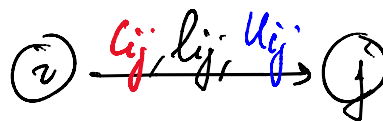
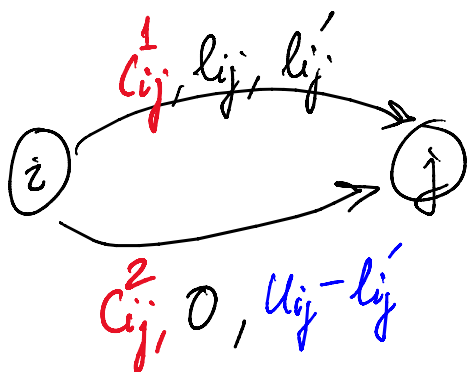
SP tree

Sufficient to show $d(i)$ and the SP tree. No need to show any other work.

4. (15) The cost incurred on flow x_{ij} on arc (i, j) in a min-cost flow network is defined as follows.

$$c_{ij}(x_{ij}) = \begin{cases} c_{ij}^1 x_{ij} & \text{if } l_{ij} \leq x_{ij} \leq l'_{ij}, \\ c_{ij}^1 l'_{ij} + c_{ij}^2 (x_{ij} - l'_{ij}) & \text{if } l'_{ij} \leq x_{ij} \leq u_{ij}, \end{cases}, \text{ where } l_{ij} \leq l'_{ij} \leq u_{ij} \text{ and } c_{ij}^1 \leq c_{ij}^2.$$

How will you model this problem as a standard min-cost flow problem?



Since $c_{ij}^1 \leq c_{ij}^2$, the first arc is saturated before any flow goes through the second arc.

6. Can find s-p SP and p-t SP by two calls of Dijkstra. $O(n^2)$ if using default implementation. The join the two SPs to get a walk, which need not be a path.

